# Azure Service Bus – Complete a Message Using C

## 1. Objective

In this example, we will learn how to:

1. Send a message to an Azure Service Bus Queue.
2. Read the message using C#.
3. Mark the message as **Complete**.
4. Verify that the completed message is removed from the queue.

### Main Concept

> When a receiver successfully completes a message, Azure Service Bus removes that message from the queue.

---

# 2. Step 1 – Send a Message to the Queue

First, open the Azure Portal and go to:

**Service Bus → Queue → Service Bus Explorer**

Initially, the queue contains no messages.

```
Active Messages = 0
```

Send a test message:

```
Message to Mark as completed
```

After sending the message, the queue contains:

```
Active Messages = 1
```

We can use **Pick from start** in Service Bus Explorer to verify that the message exists.

---

# 3. Step 2 – Create Service Bus Client

In the C# application, we first need the Service Bus connection string and queue name.

```
string connectionString = "YOUR_CONNECTION_STRING";
string queueName = "YOUR_QUEUE_NAME";
```

Create the `ServiceBusClient`:

```
ServiceBusClient client =
    new ServiceBusClient(connectionString);
```

### What is ServiceBusClient?

`ServiceBusClient` is used to establish communication with Azure Service Bus.

---

# 4. Step 3 – Create the Receiver

Create a receiver using the Service Bus client and queue name.

```
ServiceBusReceiver receiver =
    client.CreateReceiver(queueName);
```

The receiver is responsible for receiving messages from the queue.

### Flow

```
Connection String
        ↓
ServiceBusClient
        ↓
ServiceBusReceiver
        ↓
Queue
```

---

# 5. Step 4 – Receive the Message

Use `ReceiveMessageAsync()` to read a message from the queue.

```
ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();
```

Now the message is stored in the `message` variable.

For example:

```
message.Body
    ↓
"Message to Mark as completed"
```

---

# 6. Step 5 – Mark the Message as Complete

Once the application successfully processes the message, we can mark it as complete.

Use:

```
await receiver.CompleteMessageAsync(message);
```

### What does this do?

It tells Azure Service Bus:

> "I have successfully processed this message. You can remove it from the queue."

---

# 7. Complete Code Example

```
using Azure.Messaging.ServiceBus;

class Program
{
    static async Task Main(string[] args)
    {
```

```
        string connectionString = "YOUR_CONNECTION_STRING";
        string queueName = "YOUR_QUEUE_NAME";

        ServiceBusClient client =
            new ServiceBusClient(connectionString);

        ServiceBusReceiver receiver =
            client.CreateReceiver(queueName);

        ServiceBusReceivedMessage message =
            await receiver.ReceiveMessageAsync();

        Console.WriteLine(message.Body.ToString());

        await receiver.CompleteMessageAsync(message);

        Console.WriteLine("Message completed successfully.");
    }
}
```

---

# 8. What Happens Internally?

Before completing the message:

```
Azure Service Bus Queue
        ↓
Message exists
        ↓
"Message to Mark as completed"
```

The application receives it:

```
Queue
  ↓
ReceiveMessageAsync()
  ↓
message variable
```

Then the application completes it:

```
CompleteMessageAsync(message)
        ↓
Message successfully processed
        ↓
Message removed from queue
```

After completion:

```
Queue
  ↓
No message
```

---

# 9. Practical Verification in Azure Portal

We can verify the operation directly from Azure Portal.

### Before Complete

The queue contains:

```
Active Messages = 1
```

Using **Pick from start**, we can see:

```
Message to Mark as completed
```

### After Complete

Execute:

```
await receiver.CompleteMessageAsync(message);
```

Refresh the Service Bus Explorer.

Now:

```
Active Messages = 0
```

The message has disappeared from the queue.

---

# 10. Important Point – Complete Does NOT Mean Read

Receiving a message and completing a message are two different operations.

### Receive

```
var message =
    await receiver.ReceiveMessageAsync();
```

Means:

> Give me a message from the queue.

### Complete

```
await receiver.CompleteMessageAsync(message);
```

Means:

> I have successfully processed this message. Remove it from the queue.

So the flow is:

```
Receive
   ↓
Process
   ↓
Complete
   ↓
Message removed
```

---

# 11. Real-Time Example

Imagine an **Order Processing Service**.

A queue contains:

```
Order #1001
```

Our application receives the order:

```
var message =
    await receiver.ReceiveMessageAsync();
```

The application processes the order:

```
Validate Order
      ↓
Process Payment
      ↓
Create Order
      ↓
Success
```

After successful processing:

```
await receiver.CompleteMessageAsync(message);
```

The message is removed from the queue.

### Why?

Because the application has successfully processed the message and no longer needs it.

---

# 12. What If Processing Fails?

We should **not** complete the message if processing fails.

For example:

```
Receive Message
      ↓
Process Message
      ↓
```

```
Processing Failed
      ↓
Do NOT Complete
```

Depending on the processing strategy, we may abandon, defer, or dead-letter the message.

For example:

```
await receiver.AbandonMessageAsync(message);
```

This allows the message to become available for processing again.

---

# 13. Complete vs Abandon

| Operation | Result |
|---|---|
| `CompleteMessageAsync()` | Message is removed |
| `AbandonMessageAsync()` | Message becomes available again |
| `DeferMessageAsync()` | Message is kept aside for later |
| `DeadLetterMessageAsync()` | Message moves to Dead-Letter Queue |

### Easy Memory Trick

```
Complete  → Done → Delete

Abandon   → Try Again

Defer     → Process Later

Dead-Letter → Problem → Investigate
```

---

# 14. Important Code to Remember

### Create Client

```
ServiceBusClient client =
    new ServiceBusClient(connectionString);
```

### Create Receiver

```
ServiceBusReceiver receiver =
    client.CreateReceiver(queueName);
```

### Receive Message

```
ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();
```

### Complete Message

```
await receiver.CompleteMessageAsync(message);
```

The most important line in this example is:

```
await receiver.CompleteMessageAsync(message);
```

It tells Azure Service Bus that the message has been successfully processed and can be removed from the queue.

---

# 15. Interview Question

### Q: How do you mark a message as completed in Azure Service Bus?

**Answer:**

After successfully processing the received message, we call:

```
await receiver.CompleteMessageAsync(message);
```

This tells Azure Service Bus that the message has been successfully processed, so the message is removed from the queue.

---

# 16. Interview Question

### Q: What happens when a message is completed?

**Answer:**

When a receiver completes a message successfully, Azure Service Bus removes that message from the queue. The message will no longer be available for normal processing.

---

# 17. Interview Question

### Q: Should we complete a message if processing fails?

**Answer:**

No. We should complete a message only after successful processing. If processing fails, we can use appropriate handling such as abandoning, deferring, or dead-lettering the message depending on the scenario.

---

# 18. Quick Revision

```
1. Send message to Queue
          ↓
2. Receive message
          ↓
3. Process message
          ↓
4. Processing successful?
          ↓
         YES
          ↓
5. CompleteMessageAsync()
          ↓
6. Message removed from Queue
```

## One-Line Definition

**Complete Message = Successfully processed message → Remove it from the Azure Service Bus Queue.**